S11 — Integración del cliente: frontend Angular 22 sobre el Gateway, con Keycloak

La veterinaria deja su servidor de autorización propio (`veterinaria-auth-ms`) y pasa a Keycloak (S07b). Sobre esa base, una SPA en Angular 22 inicia sesión con Authorization Code + PKCE, consume los microservicios solo a través del Gateway (CORS centralizado) y muestra con honestidad el resultado asíncrono de la Saga de pago de S9: una cita recién creada está *pendiente* hasta que `veterinaria-pago-ms` la valida o la rechaza.

1. Datos del estudiante
2. Introducción y motivación
3. Arquitectura
4. Bloque 1 — Migración a Keycloak
5. Bloque 2 — Authorization Code + PKCE
6. Bloque 3 — Proceso asíncrono en la interfaz
7. Bloque 4 — Camino feliz y caminos de fallo
8. Error o hallazgo técnico
9. Preguntas de defensa
10. Reflexión técnica
11. Criterios, limitaciones y bibliografía
12. Anexo: Feedback de la sesión

ESTUDIANTE
Joaquín Hassan Sanchez Saravia

SESIÓN
S11 · Integración con cliente frontend

FECHA
22 de octubre de 2026

Datos del estudiante

Nombre	Joaquín Hassan Sanchez Saravia
Equipo	##
Sesión	S11 — Integración con Cliente Frontend: Angular 22 sobre el Gateway, con Keycloak
Rol o aporte realizado	Implementación individual completa: S07b (Keycloak en Docker con realm importado, user attribute + protocol mapper `idCliente`, migración de los cuatro resource servers a `issuer-uri`, retiro de `veterinaria-auth-ms`), CORS en el Gateway, endpoint `/api/v1/pagos/por-cita/{citaId}`, y la SPA `clients/veterinaria-frontend` (PKCE, guards, interceptores, especies, nueva cita, "Mis citas" con polling y pagos para ADMIN), con pruebas y evidencia.
Link de GitHub	https://github.com/JoaHasss/veterinaria-mascota-ms
Commits	`a4a0c28` S07b — Keycloak reemplaza a veterinaria-auth-ms (22/10/2026 15:10) `5bdde6d` S11 — Integración del cliente: frontend Angular 22 con Keycloak (22/10/2026 15:10)

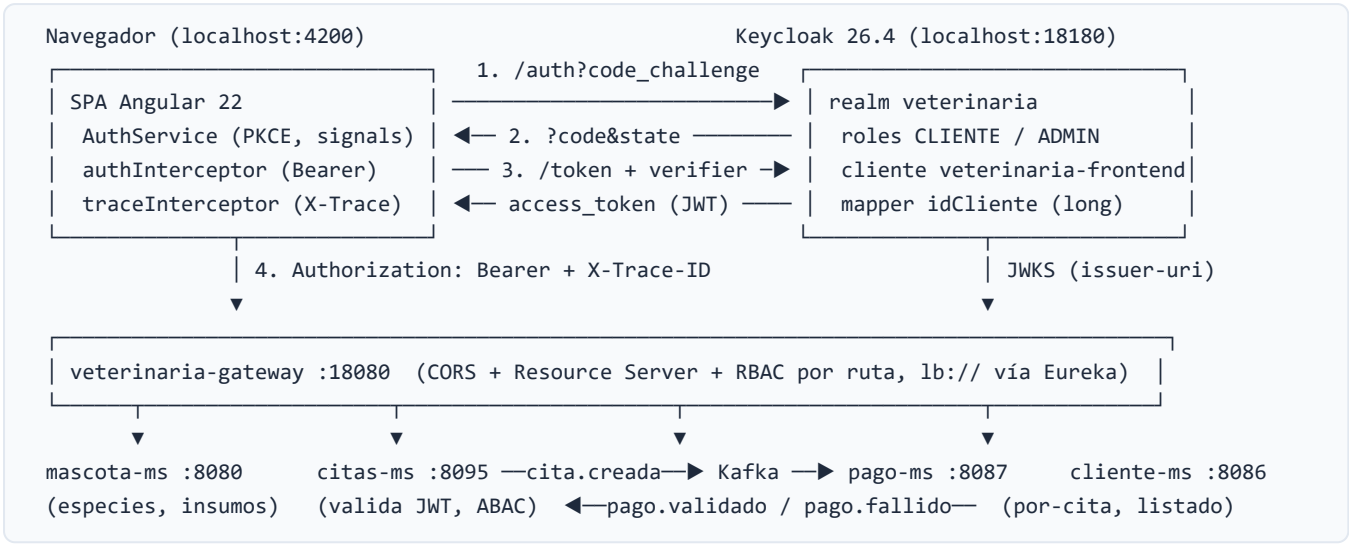
> **Sobre la evidencia.** Las capturas son de pantalla completa (Chrome maximizado y barra de tareas de Windows con el reloj) y se tomaron el 22/10/2026 entre las 15:23 y las 15:27 (hora de Lima, UTC−5), después de los commits. Las salidas de consola y la bitácora son copia literal de esa misma ejecución; cada línea lleva su marca de tiempo.

01 Introducción y motivación

Hasta S9 todo el sistema se probó con `curl` y Postman: el token se pedía con usuario y contraseña a `veterinaria-auth-ms` y se pegaba a mano en cada petición. Un cliente real no puede funcionar así. Un navegador no guarda secretos, habla con otro origen (el Gateway) y además recibe respuestas que todavía no son definitivas, porque el pago de una cita lo resuelve otro servicio por Kafka.

S11 resuelve tres problemas. El primero es la identidad: el servidor de autorización propio se reemplaza por Keycloak, y la SPA obtiene el token con Authorization Code + PKCE, sin ver nunca la contraseña. El segundo es el origen: el navegador solo habla con el Gateway, que concentra CORS. El tercero es la consistencia: un `201` de `POST /api/v1/citas` solo dice que la cita existe, no que se pagó. La interfaz tiene que mostrar "todavía no se sabe" hasta que la Saga de S9 termine.

02 Arquitectura



Los pasos 1 a 3 son el flujo Authorization Code + PKCE: la SPA nunca recibe la contraseña y el código solo se puede canjear con el `code_verifier` que quedó en el navegador. Desde el paso 4, cada petición va al Gateway, que valida firma, expiración e issuer contra Keycloak y aplica RBAC por ruta. `citas-ms` y `veterinaria-cliente-ms` vuelven a validar el token (defensa en profundidad) y aplican ABAC con el claim `idCliente`.

Componente	Puerto	Cambio en S07b / S11
Keycloak + Postgres	18180 / 19000 / 15438	Nuevo: `infra/keycloak/compose-dev.yml` y `realm/veterinaria-realm.json` (import al arrancar)
veterinaria-gateway	18080	`issuer-uri`, sin ruta `/api/v1/auth/**`, `CorsConfig` + `.cors()`, `GET /api/v1/pagos` solo ADMIN
citas-ms, cliente-ms, pago-ms	8095 / 8086 / 8087	`issuer-uri` del realm; pago-ms agrega `/por-cita/{citaId}`
veterinaria-auth-ms	8085	Retirado: sin ruta en el Gateway y sin arrancar. El código queda como referencia.
veterinaria-frontend	4200	Nuevo: `clients/veterinaria-frontend` (Angular 22.2, Node 24.22)

03 Bloque 1 — Migración a Keycloak (S07b)

Keycloak corre en Docker con su propia base Postgres. El puerto de la guía (15436) ya lo usaba la base de `veterinaria-cliente-ms`, por eso Keycloak usa 15438. En lugar de configurar el realm a mano en la consola, el realm completo se versiona en `infra/keycloak/realm/veterinaria-realm.json` y se importa con `start-dev --import-realm`: roles, cliente, mapper, perfil de usuario y usuarios semilla. Así cualquier integrante del equipo obtiene el mismo realm con un solo `docker compose up`.

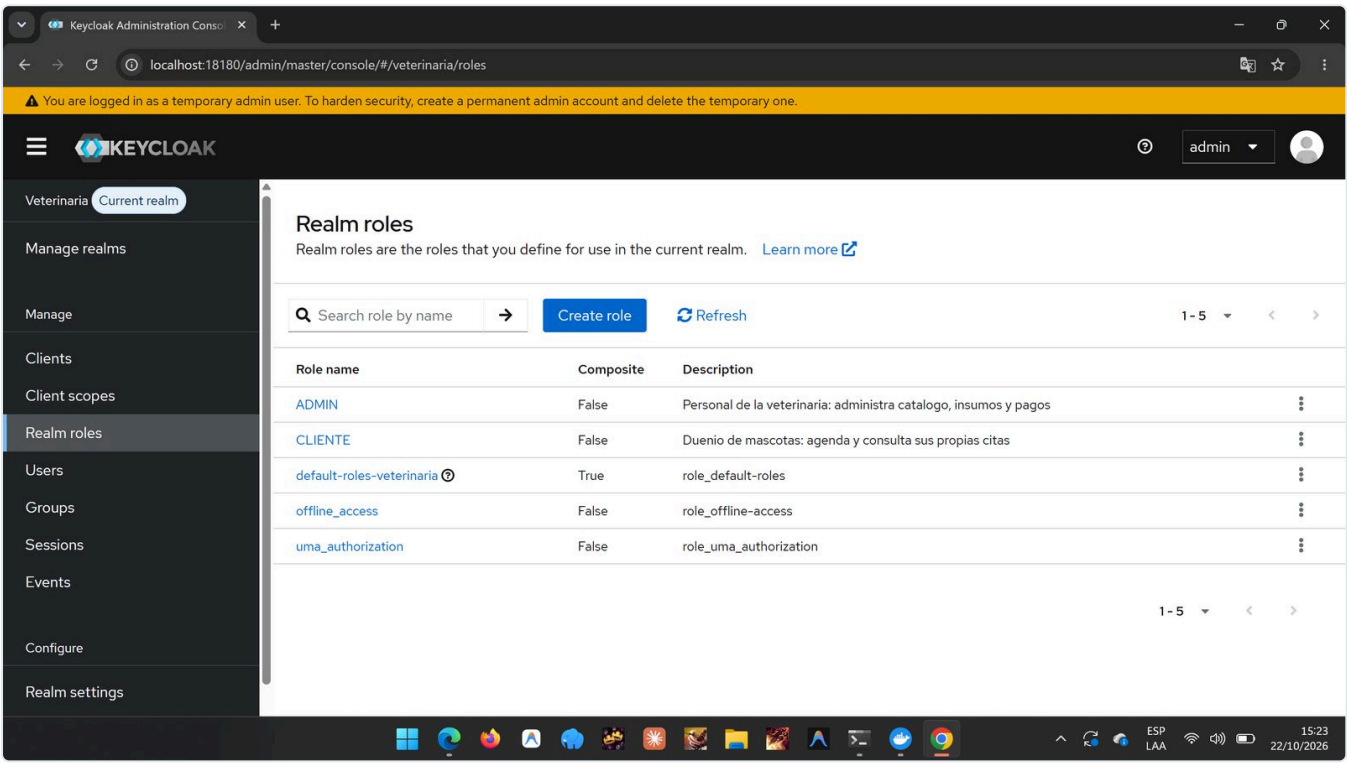


Figura 1. Realm `veterinaria` con los realm roles `CLIENTE` y `ADMIN` (los mismos nombres que ya leía `JwtRoleConverter` desde S7).

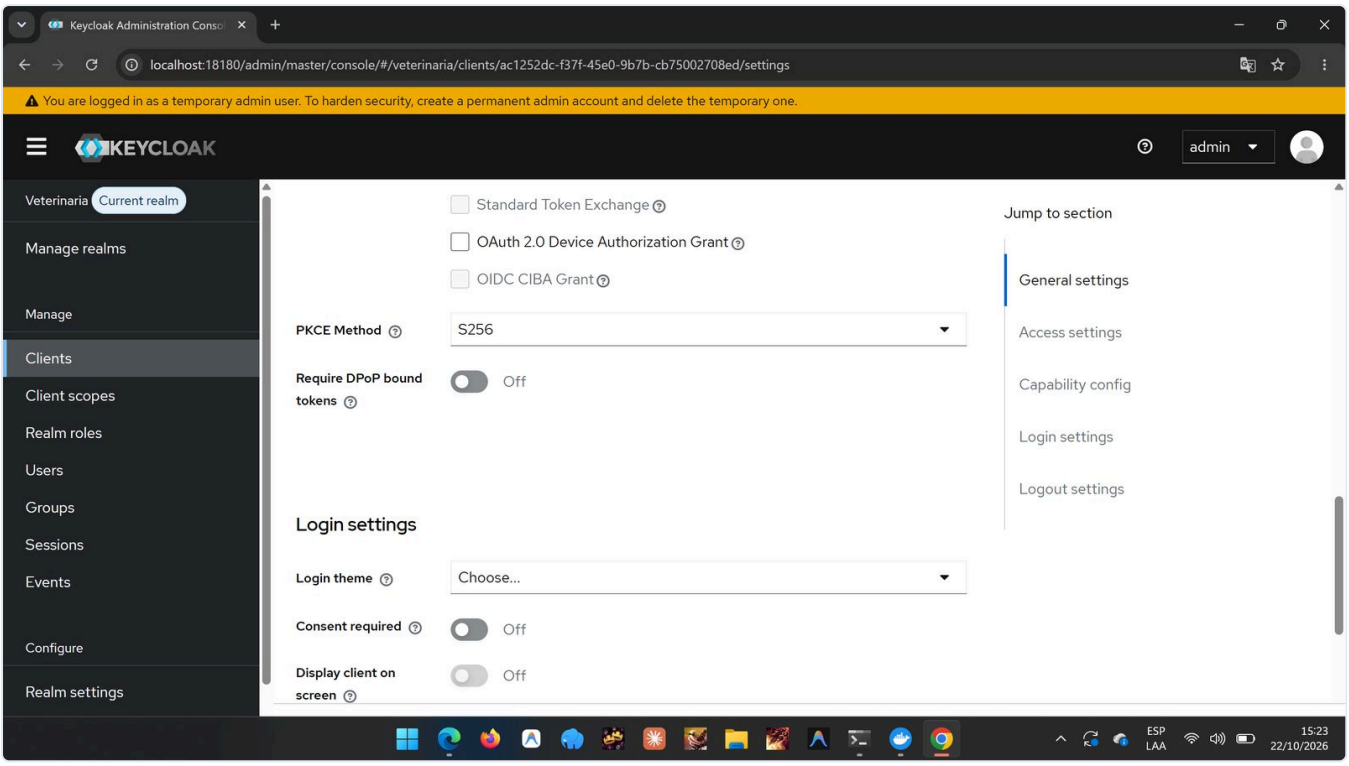


Figura 2. Cliente `veterinaria-frontend`: cliente público (sin Client authentication), solo Standard flow, sin Direct access grants y con **PKCE Method = S256** obligatorio.

Decisión 4.1.2: ¿el dato de negocio puede ser `sub`?

En pagatu, S07b reemplazó `idCliente` por `sub` porque allí la orden solo necesitaba saber "quién inició sesión". En la veterinaria no es así. `idCliente` es la clave del perfil en `veterinaria-cliente-ms` (`clientes.id`, un `BIGINT`): lo usan `GET /api/v1/clientes/me`, la política ABAC `ClienteAccessPolicy` y la columna `citas.id_cliente`, que relaciona cada cita con su dueño. `sub` identifica la *cuenta* en Keycloak (un UUID que Keycloak genera), no al *cliente* de la veterinaria. Además, una cuenta ADMIN no tiene perfil de cliente, y un cliente podría cambiar de cuenta sin dejar de ser el mismo cliente.

Por eso se eligió **user attribute + protocol mapper**: el atributo `idCliente` se declara en el perfil de usuario (solo lo edita un admin y se valida como entero ≥ 1), y un mapper *User Attribute* lo emite como claim `long` en el access token. Con esta decisión no hubo que migrar columnas ni cambiar tipos en `citas-ms`, `pago-ms` ni en los eventos de Kafka. El contrato del token que esos servicios ya leían (`idCliente` numérico y `realm_access.roles`) se mantiene igual.

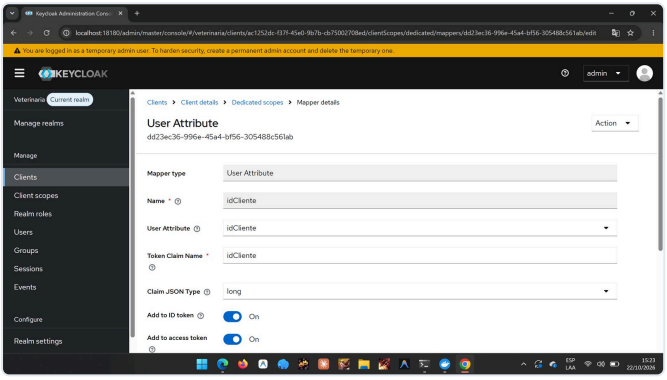


Figura 3. Mapper `idCliente`: User Attribute → claim `idCliente`, tipo `long`, en ID token y access token.

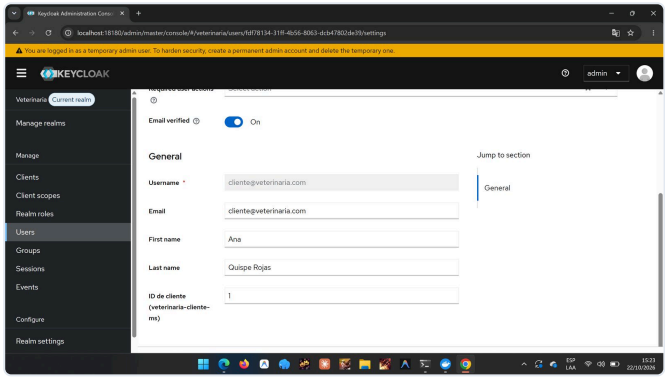


Figura 4. Usuario `cliente@veterinaria.com` con el atributo `idCliente` = 1 (el mismo id de su perfil en cliente-ms).

Verificación con un token real

Payload de un access token obtenido con Authorization Code + PKCE para `cliente@veterinaria.com` (22/10/2026 15:04):

```
{
  "iss": "http://localhost:18180/realms/veterinaria",
  "sub": "fdf78134-31ff-4b56-8063-dcb47802de39",       ← cuenta Keycloak (UUID)
  "azp": "veterinaria-frontend",
  "allowed-origins": [ "http://localhost:4200" ],
  "realm_access": { "roles": [ "CLIENTE" ] },         ← RBAC (JwtRoleConverter)
  "scope": "openid profile email",
  "idCliente": 1,                                       ← protocol mapper (long, no string)
  "preferred_username": "cliente@veterinaria.com",
  "email": "cliente@veterinaria.com"
}
```

Con Direct access grants desactivado, el flujo inseguro de contraseña queda cerrado para la SPA:

```
$ curl -d client_id=veterinaria-frontend -d grant_type=password -d username=cliente@veterinaria.com ... /to
ken
{"error":"unauthorized_client","error_description":"Client not allowed for direct access grants"}
```

Resource servers con `issuer-uri`

En el Config Server, los perfiles dev y prod del Gateway, `citas-ms`, `veterinaria-cliente-ms` y `veterinaria-pago-ms` pasan de `jwk-set-uri` (JWKS de auth-ms) a `issuer-uri`. Con `issuer-uri` Spring descubre el JWKS en el documento `.well-known/openid-configuration` del realm y además rechaza tokens cuyo `iss` no sea exactamente ese realm.

```
# infra/veterinaria-config/config-repo/veterinaria-gateway-dev.yml (igual en citas-ms, cliente-ms y pago-ms)
spring.security.oauth2.resourceserver.jwt:
  issuer-uri: http://localhost:18180/realms/veterinaria
# perfil prod: por nombre de servicio Docker
  issuer-uri: http://veterinaria-keycloak:8080/realms/veterinaria
```

Pruebas por el Gateway con tokens reales de Keycloak (22/10/2026 15:06):

Petición	Token	Esperado	Obtenido
`POST /api/v1/especies`	CLIENTE	403 (sin rol)	**403 · insufficient_scope**
`POST /api/v1/especies`	sin token	401	**401**
`GET /api/v1/especies`	CLIENTE	200	**200**
`GET /api/v1/clientes/me`	CLIENTE	200, perfil id 1	**200 · Ana Quispe Rojas**
`GET /api/v1/citas`	CLIENTE	200 (solo las propias)	**200**
`GET /api/v1/pagos`	CLIENTE	403	**403**
`GET /api/v1/pagos`	ADMIN	200	**200**
`POST /api/v1/citas` (YAPE)	CLIENTE	201, `idCliente` = 1	**201 · cita #13 · idCliente 1**

La distinción es la de S7: **401** significa que el Gateway no pudo validar el token (no hay token, expiró o lo firmó otro issuer); **403** significa que el token es válido pero el rol no alcanza.

Retiro de `veterinaria-auth-ms`. Se quitó su ruta `/api/v1/auth/**` del Gateway y el `permitAll` asociado, y el servicio ya no se levanta. Se conservó su base de datos (no se ejecutó `down -v`) y su código como referencia.

04 Bloque 2 — Authorization Code + PKCE en la SPA

La SPA no usa librerías OIDC: el flujo está en tres archivos pequeños para que cada verificación quede a la vista. `pkce.ts` genera con Web Crypto un `code_verifier` de 32 bytes, su `code_challenge` (SHA-256 en Base64URL) y un `state` de 16 bytes. `AuthService.iniciarLogin()` guarda verifier, state y la ruta de destino en `sessionStorage` (solo durante el viaje a Keycloak) y redirige a `/protocol/openid-connect/auth` con `code_challenge_method=S256`.

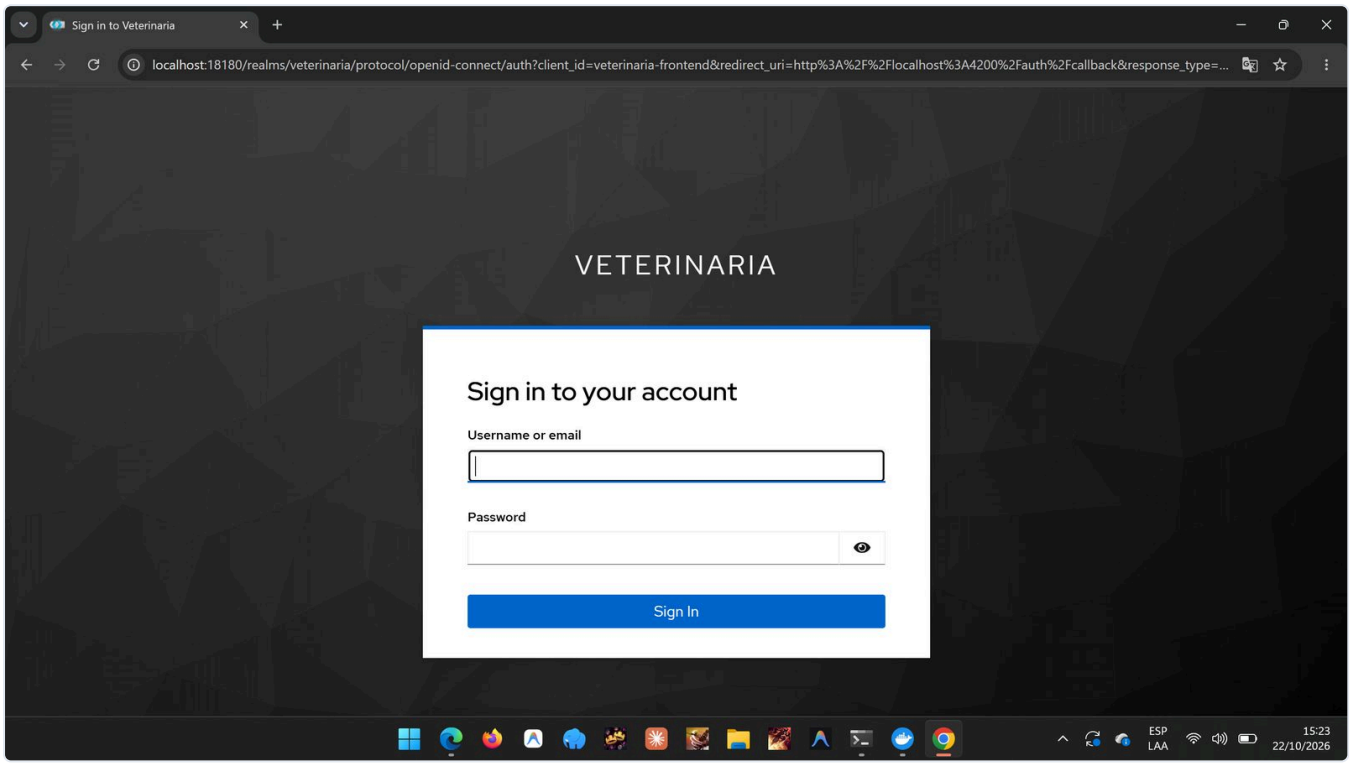


Figura 5. Pantalla de login del realm `veterinaria`. La contraseña se escribe en Keycloak, no en la SPA.

URL real de esa pantalla (bitácora, 15:23:55). Solo viaja el *challenge*; el verifier nunca sale del navegador hasta el canje:

```
http://localhost:18180/realms/veterinaria/protocol/openid-connect/auth?client_id=veterinaria-frontend
  &redirect_uri=http%3A%2F%2Flocalhost%3A4200%2Fauth%2Fcallback&response_type=code&scope=openid+profile
  &state=q_4PXBHMWmkWsy_yn5Gk-A
  &code_challenge=u0ZKFEJ5AfY3WW9FPYwKSyhbG12a7jHvevTCBwIqL7k&code_challenge_method=S256
```

Al volver a `/auth/callback?code=…&state=…`, `completarLogin()` compara el `state` recibido con el guardado **antes** de canjear el código, borra los valores de `sessionStorage` y envía el `code_verifier` a `/token`. Usa `fetch` y no `HttpClient` a propósito, para que esta llamada a Keycloak no pase por los interceptores pensados para el Gateway.

```
// clients/veterinaria-frontend/src/app/core/auth/auth-service.ts (extracto)
if (!stateRecibido || stateRecibido !== stateEsperado) {
  console.warn('[auth] state NO coincide: se descarta la respuesta', { stateRecibido, stateEsperado });
  throw new Error('El parametro state no coincide: posible CSRF, se descarta la respuesta.');
}
console.info('[auth] state verificado', stateRecibido);
const respuesta = await fetch(`${this.endpoint}/token`, { method: 'POST', body: new URLSearchParams({
  grant_type: 'authorization_code', client_id: this.kc.clientId, code,
  redirect_uri: this.kc.redirectUri, code_verifier: verifier }) });
```

Consola del navegador en la misma ejecución. El `state` verificado es el mismo que viajó en la URL de la página anterior:

```
22/10/2026, 15:23:59  consola: [auth] state verificado q_4PXBHMWmkWsy_yn5Gk-A
22/10/2026, 15:23:59  consola: [auth] code canjeado con code_verifier; token valido por 900 s
```

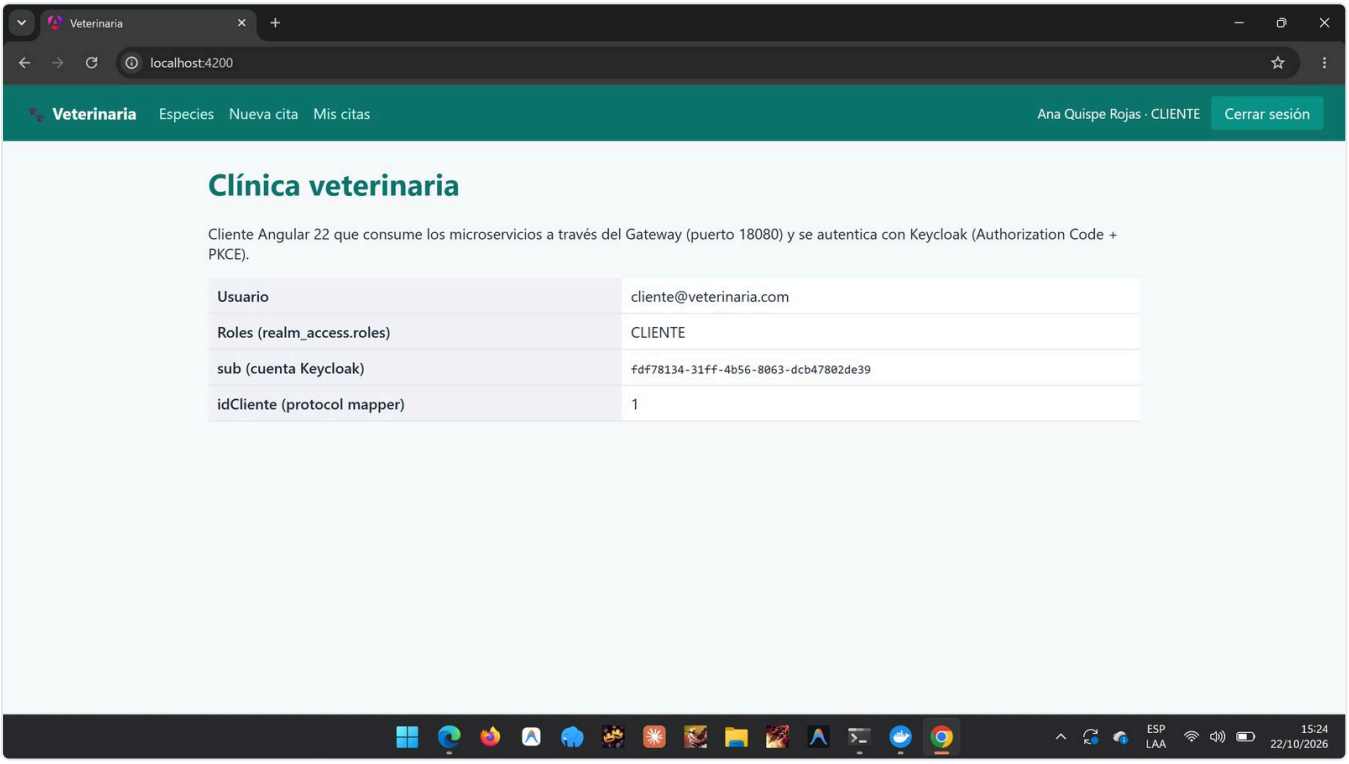


Figura 6. Sesión iniciada: la página de inicio muestra los claims leídos del token (roles, `sub` e `idCliente` del mapper).

Dónde vive el token. El access token y el ID token se guardan en *signals* en memoria, nunca en `localStorage`, donde cualquier script inyectado por XSS los podría leer. Al recargar la página el token se pierde y `authGuard` vuelve a iniciar el login. Como la sesión SSO de Keycloak sigue activa, el usuario no vuelve a escribir su contraseña: el viaje de ida y vuelta es automático (Figura 13).

Interceptores, guards y CORS

Pieza	Qué hace
`authInterceptor`	Agrega `Authorization: Bearer` solo a URLs del Gateway. Ante un 401 reinicia el login; el 403 lo muestra cada pantalla.
`traceInterceptor`	Agrega `X-Trace-ID: crypto.randomUUID()`. Los `CorrelationIdFilter` de S6 lo ponen en el MDC de cada servicio.
`authGuard`	Sin sesión, guarda la ruta pedida e inicia el login; al volver retoma esa ruta.
`rolGuard('ADMIN')`	Oculta pantallas por rol (UX). El límite real sigue siendo el Gateway.
`app.config.ts`	`provideHttpClient(withInterceptors([authInterceptor, traceInterceptor]))`: sin esto no sale ningún token.

El Gateway es Spring Cloud Gateway **Server WebMVC** (servlet), así que `spring.cloud.gateway.globalcors` no aplica. Se publica un `CorsConfigurationSource` (`CorsConfig.java`) con el origen leído de `veterinaria.cors.allowed-origins` y se activa con `.cors(cors -> {})` **antes** de las reglas de autorización. Si no se hace así, el preflight `OPTIONS`, que nunca lleva token, cae en `anyRequest().authenticated()` y responde 401.

```
$ curl -i -X OPTIONS http://localhost:18080/api/v1/citas -H "Origin: http://localhost:4200" \
       -H "Access-Control-Request-Method: POST"                    # 22/10/2026 15:06 → 200
Access-Control-Allow-Origin: http://localhost:4200
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
Access-Control-Expose-Headers: X-Trace-ID
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 3600
```

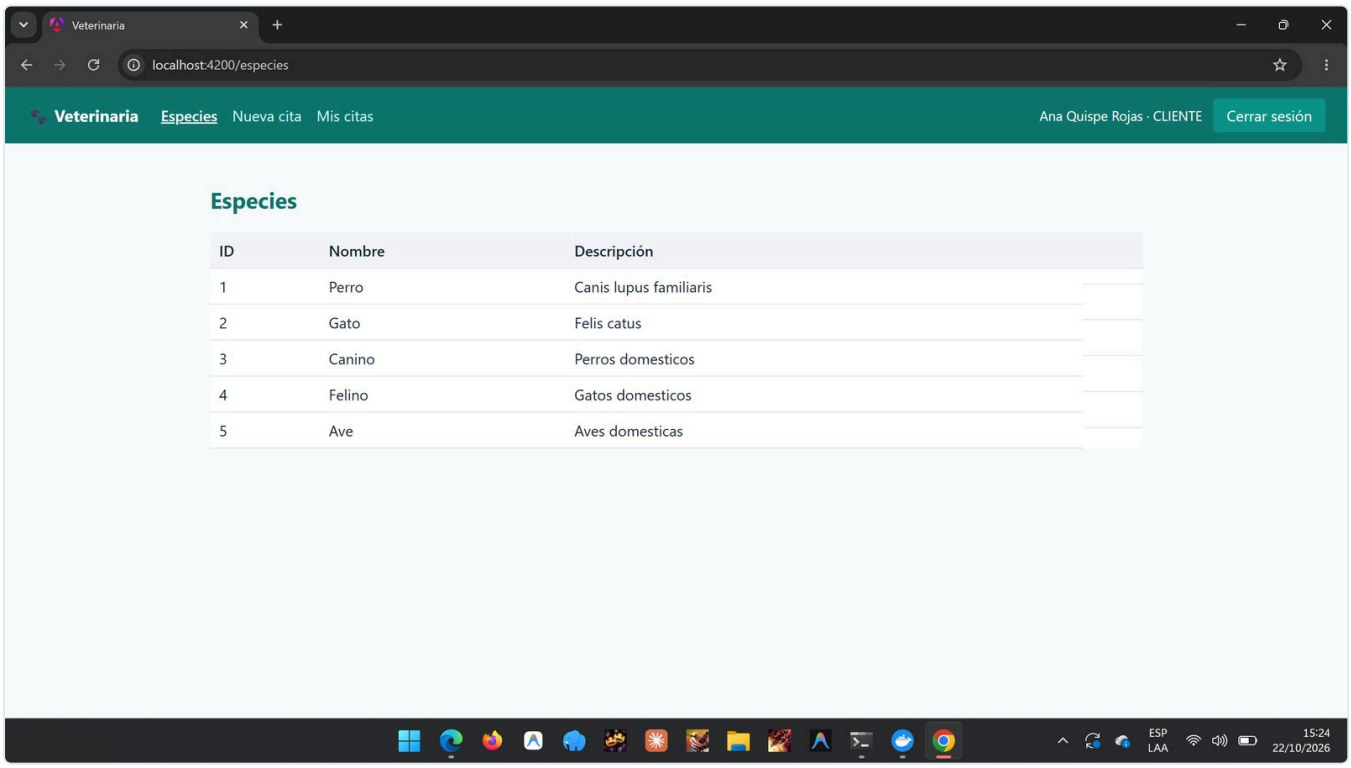


Figura 7. Especies a través del Gateway con el Bearer de Keycloak. Para CLIENTE no se muestran los botones de edición; para ADMIN se habilita el CRUD (crear, editar, eliminar).

05 Bloque 3 — Proceso asíncrono en la interfaz

La pantalla asíncrona es **"Mis citas"**. Cuando el cliente agenda una cita, `citas-ms` responde `201` con estado `PENDIENTE_PAGO` y publica `cita.creada`. Recién después, `veterinaria-pago-ms` consume el evento, decide y publica `pago.validado` o `pago.fallido`. Entonces `citas-ms` pasa la cita a `PAGADA` o la compensa a `CANCELADA` (Saga de S9). Entre el 201 y ese momento, la respuesta honesta es "todavía no se sabe".

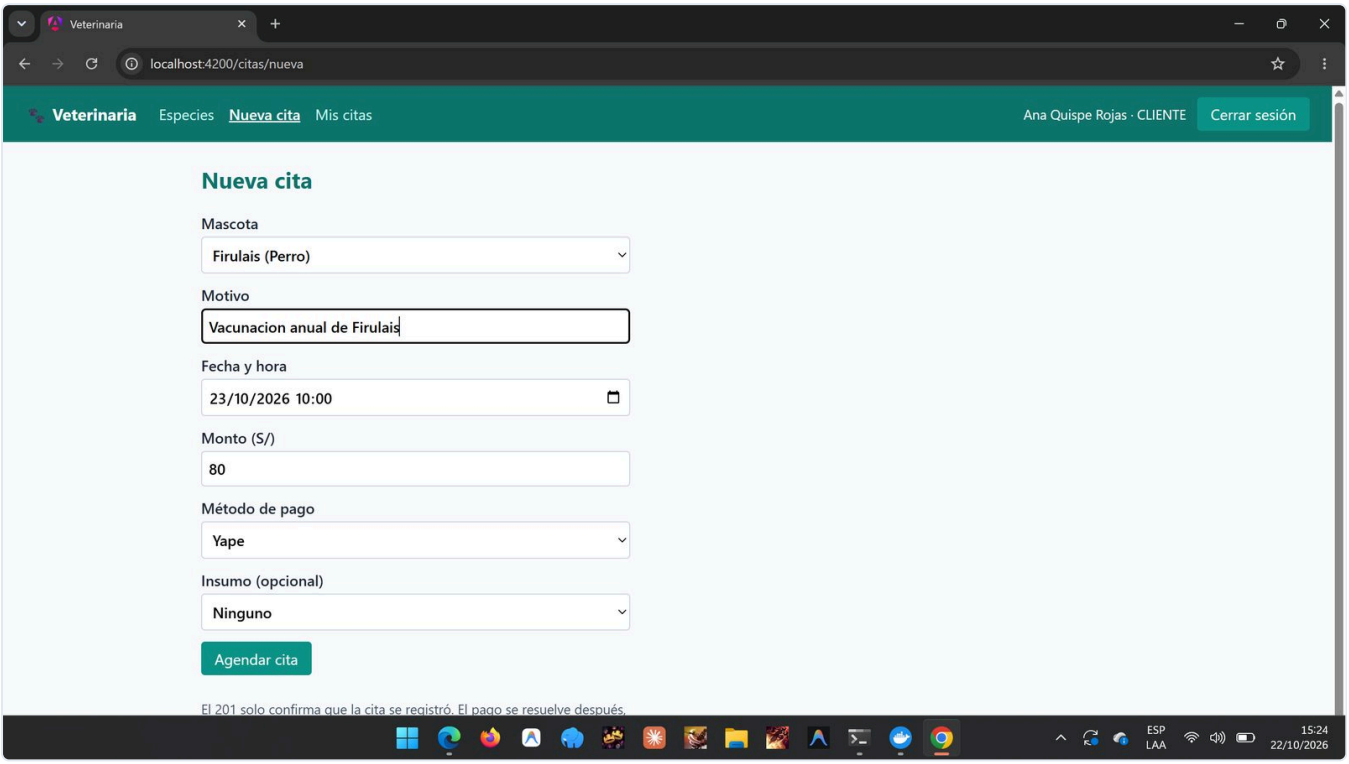


Figura 8. Formulario "Nueva cita". La nota bajo el botón advierte que el 201 no confirma el pago. El `idCliente` no se envía: `citas-ms` lo toma del token.

Estrategia elegida: polling acotado

Estrategia (Tabla 3)	Decisión para la veterinaria
Polling (elegida)	Funciona con lo que ya existe: `GET /api/v1/citas/{id}` por el Gateway, con el mismo token, RBAC y ABAC. La Saga tarda menos de un segundo con pago-ms activo, así que un intervalo de 4 s cuesta pocas peticiones, y solo se consultan las citas `PENDIENTE_PAGO`.
Push (SSE / WebSocket)	Descartada. Necesita una conexión larga que el Gateway WebMVC y el Resource Server tendrían que autenticar y mantener, y un componente que traduzca eventos de Kafka a conexiones de cada cliente. Es mucha infraestructura para pocas citas al día.
Respuesta síncrona (esperar el pago en el POST)	Descartada. Haría que la creación de la cita dependiera de que pago-ms esté vivo, que es justo lo que la Saga de S8/S9 desacopló.

```
// clients/veterinaria-frontend/src/app/features/citas/mis-citas.ts (extracto)
const pendientes = this.items().filter((i) => i.cita.estado === 'PENDIENTE_PAGO' && !i.demorada);
...  this.citas.obtener(item.cita.id).subscribe({ next: (cita) => {
       const demorada = cita.estado === 'PENDIENTE_PAGO' && intentos >= environment.pollingMaxIntentos;  // 30 × 4 s
       if (cita.estado !== 'PENDIENTE_PAGO') this.cargarPago(cita.id);  // GET /pagos/por-cita/{id}, 404 = aún no
ngOnDestroy(): void { this.detenerSondeo(); }  // sin esto el intervalo sigue tras salir de la pantalla
```

El sondeo es **acotado**: se detiene solo cuando no quedan citas pendientes, cuando se sale de la pantalla (`ngOnDestroy`) o después de 30 intentos sin cambio (2 minutos). En el último caso la cita no se marca como fallida, porque eso sería inventar un resultado: se muestra "Demora inusual: el pago no se ha resuelto. Revisa más tarde". El pago de la cita se pide a `GET /api/v1/pagos/por-cita/{citaId}`, un endpoint nuevo en pago-ms que responde **404 cuando el pago aún no existe**. La SPA trata ese 404 como "todavía no se sabe" y no como un error.

06 Bloque 4 — Camino feliz y caminos de fallo

Camino feliz: YAPE → PAGADA

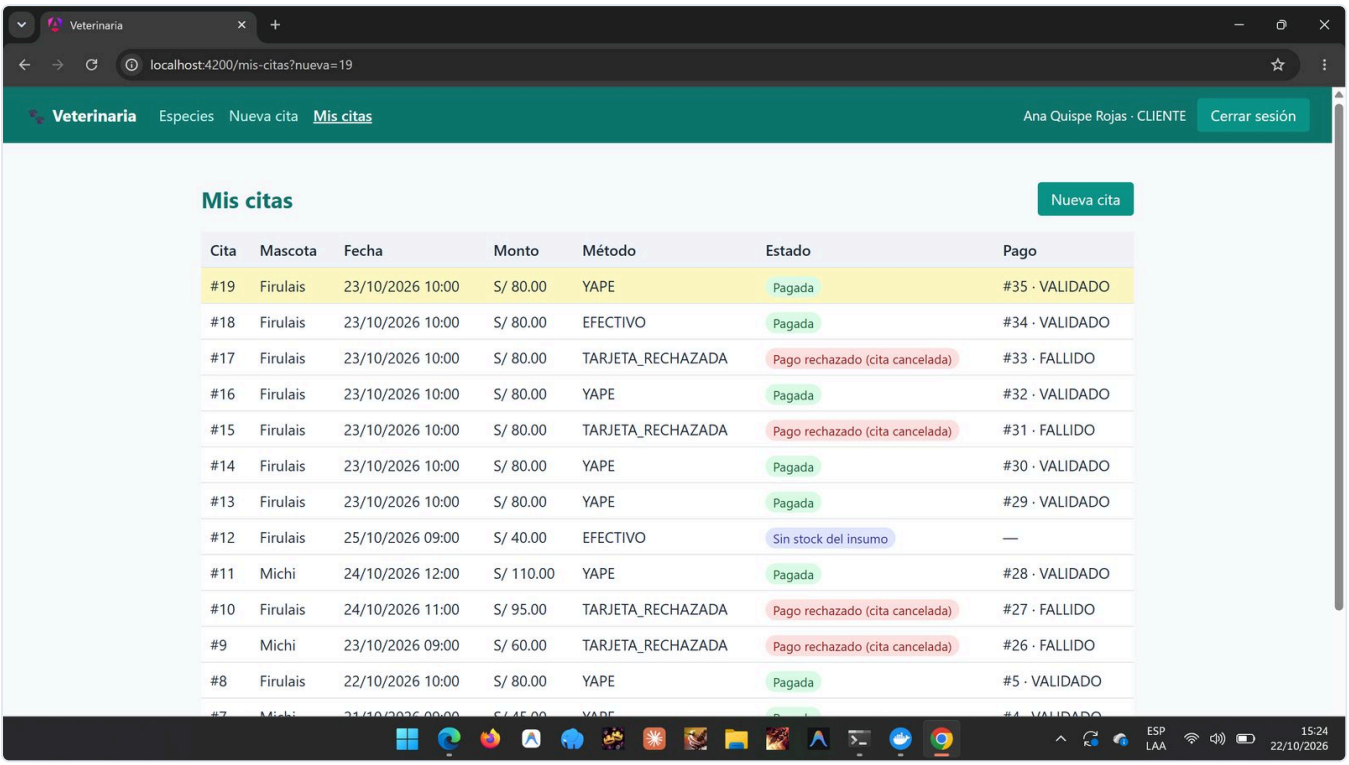


Figura 9. Cita #19 (YAPE) creada a las 15:24:11; a las 15:24:16 ya figura "Pagada" con pago VALIDADO, sin recargar la página (resaltada en amarillo).

Camino de rechazo: TARJETA_RECHAZADA → CANCELADA (compensación)

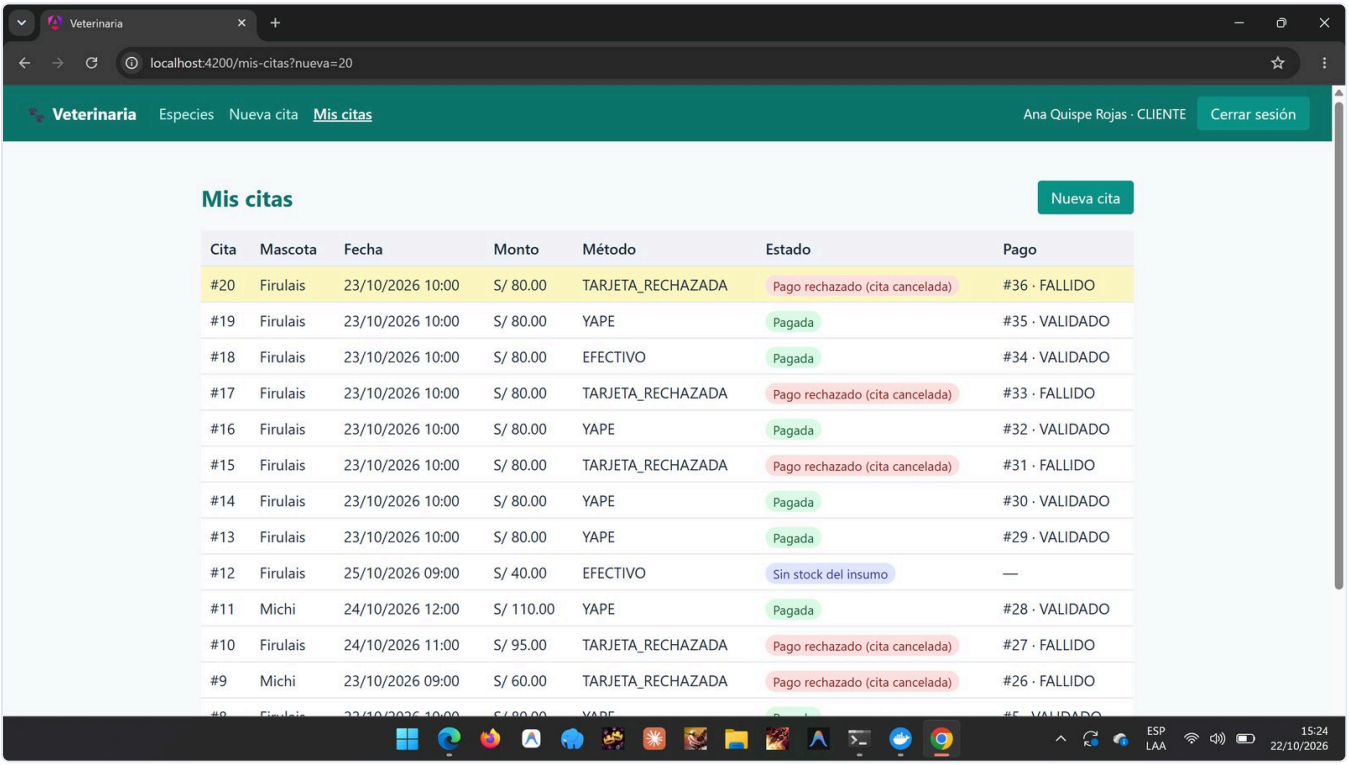


Figura 10. Cita #20 con `TARJETA_RECHAZADA`: la pasarela simulada de pago-ms la rechaza, publica `pago.fallido` y citas-ms la compensa a CANCELADA. La interfaz dice "Pago rechazado (cita cancelada)" y el pago figura FALLIDO.

```
22/10/2026, 15:24:17  POST /api/v1/citas -> 201, cita #20 (TARJETA_RECHAZADA)
22/10/2026, 15:24:21  consola: [polling] cita 20: PENDIENTE_PAGO -> CANCELADA, se deja de sondear
22/10/2026, 15:24:21  consola: [polling] detenido
```

Camino de fallo de infraestructura: pago-ms apagado

Para ver el estado intermedio sin depender de la velocidad, se detuvo `veterinaria-pago-ms` y se agendó otra cita. `citas-ms` respondió 201 igual (la Saga no depende de que pago-ms esté vivo), y la cita quedó pendiente con el evento esperando en Kafka.

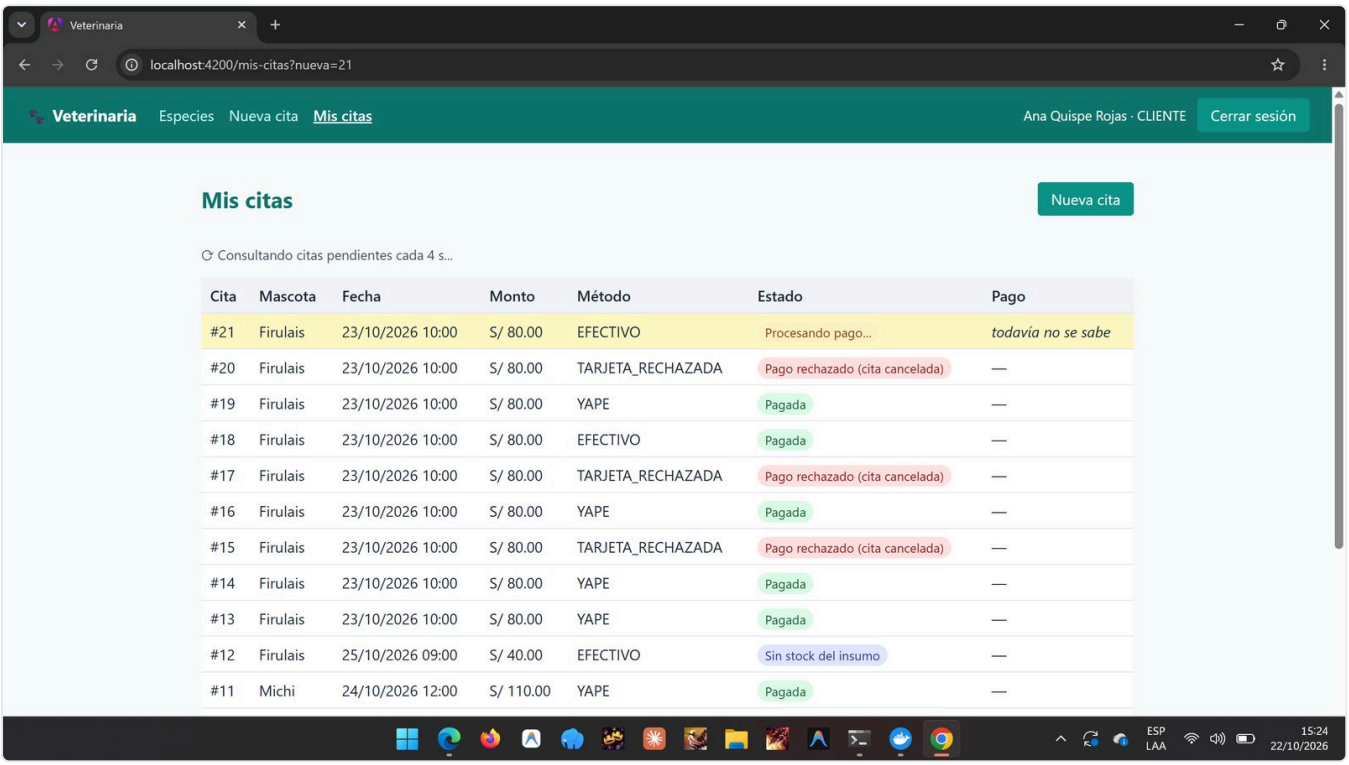


Figura 11. Cita #21 creada con pago-ms apagado: "Procesando pago...", columna Pago "todavía no se sabe" e indicador de sondeo activo. Las demás filas muestran "—" en Pago porque `/por-cita` tampoco responde mientras pago-ms está caído.

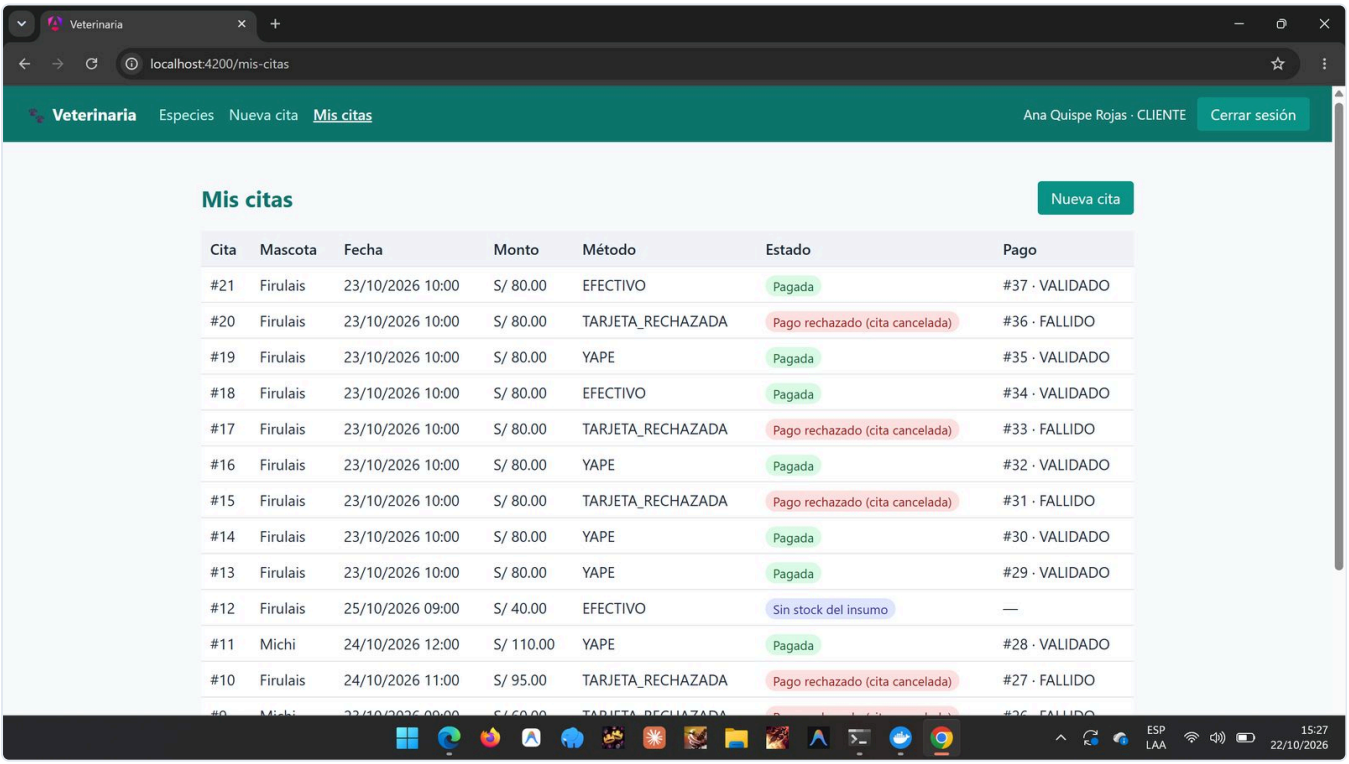


Figura 12. Tras levantar de nuevo pago-ms (15:27), este consume el `cita.creada` que quedó retenido en Kafka y, al volver a abrir "Mis citas", la cita #21 figura "Pagada" con el pago #37 VALIDADO. No se perdió el evento ni se duplicó el cobro.

Control de acceso visto desde la SPA

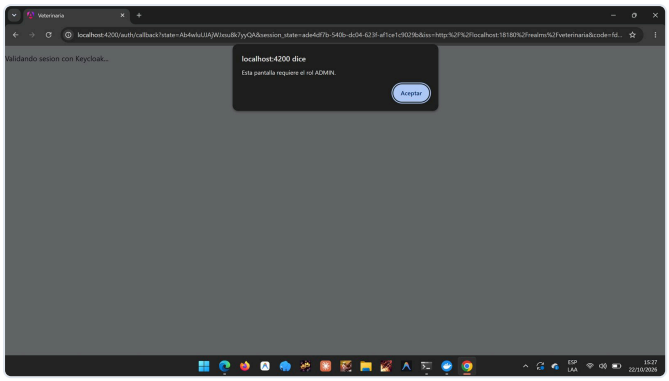


Figura 13. CLIENTE abre `/pagos` recargando la página: el token se había perdido, `authGuard` hizo el viaje SSO a Keycloak sin pedir la contraseña (URL `/auth/callback?state=…`) y luego `rolGuard('ADMIN')` lo bloqueó.

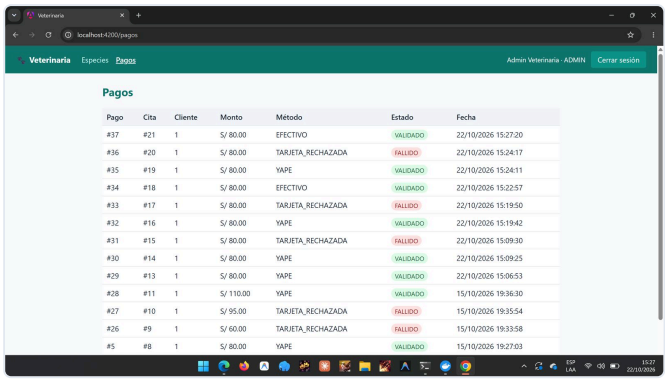


Figura 14. ADMIN ve el listado completo de pagos (VALIDADO y FALLIDO). El mismo `GET /api/v1/pagos` con token CLIENTE responde 403 en el Gateway.

Escenario	Cita	Estado final	Pago	Tiempo observado
Feliz (YAPE)	#19	PAGADA	VALIDADO	< 5 s
Rechazo (TARJETA_RECHAZADA)	#20	CANCELADA	FALLIDO	4 s
pago-ms caído y reiniciado	#21	PAGADA	VALIDADO	al volver pago-ms

07 Error o hallazgo técnico

1. El polling se quedó sin intentos mientras pago-ms no volvía

Síntoma. En la prueba de la Figura 11, el script que reiniciaba `veterinaria-pago-ms` falló (`"mvnw.cmd" no se reconoce como un comando interno o externo`, por el directorio de trabajo con espacios). La cita #21 quedó `PENDIENTE_PAGO` y la pantalla consultó 30 veces, una cada 4 s:

```
22/10/2026, 15:24:38  veterinaria-pago-ms reiniciado          ← el arranque falló
22/10/2026, 15:24:42  consola: [polling] GET /api/v1/citas/21 (intento 1)
...                                                  (28 líneas iguales)
22/10/2026, 15:26:26  consola: [polling] GET /api/v1/citas/21 (intento 30)
22/10/2026, 15:26:26  consola: [polling] detenido
```

Diagnóstico. Este es el "error frecuente" de la guía: una orden atascada en pendiente casi nunca es culpa del polling, sino de que el consumidor no está corriendo. `netstat` confirmó que nadie escuchaba en 8087. El diseño acotado se comportó como debía: no sondeó para siempre ni marcó la cita como fallida. La dejó como "Demora inusual" y dejó de cargar al Gateway. Al levantar pago-ms a mano, consumió el evento retenido y la cita pasó a PAGADA (Figura 12). En una primera versión el tope era de 15 intentos (60 s), menos que lo que tarda en arrancar un servicio Spring con Maven. Se subió a 30 (2 min), y de todos modos queda el aviso como red de seguridad.

2. "No hay pagos registrados" cuando sí había pagos

La primera versión de `PagoList` mostraba la fila vacía (`@empty`) mientras la petición seguía en curso. Una prueba automatizada leyó esa fila y concluyó que no había pagos, aunque `GET /api/v1/pagos` devolvía 200 con datos. Es el mismo problema de fondo que el de la pantalla de citas, pero al revés: la interfaz afirmaba algo ("no hay") cuando en realidad "todavía no se sabe". Se corrigió con un estado `cargando` explícito, que muestra "Cargando…" hasta que llega la respuesta.

3. Hallazgos menores de entorno

Angular CLI 22 exige Node ≥ 24.15.0. La máquina tenía 24.14.1 y se actualizó a 24.22.1 (mismo LTS). El puerto 15436 de la guía para Postgres de Keycloak ya estaba ocupado por la base de cliente-ms, así que se usó 15438. Keycloak marca sus cookies de sesión como `Secure`: un cliente HTTP de pruebas sobre `http://localhost` no las reenviaba y el login devolvía 400 hasta forzar su reenvío. El navegador sí las maneja en localhost.

08 Preguntas de defensa

1. ¿Por qué S07b decidió que `idCliente` podía ser literalmente `sub`, en vez de usar el protocol mapper diseñado en S07?

En pagatu, la orden solo necesitaba saber qué cuenta la creó, y `sub` ya es un identificador estable y único que Keycloak emite sin configurar nada. Para eso un atributo extra solo agregaba algo que mantener. En la veterinaria la respuesta es distinta (sección 3): `idCliente` referencia un perfil de negocio en otro servicio, con su propio ciclo de vida. Por eso aquí sí se usa user attribute + mapper.

2. ¿Qué problema previene verificar `state` al volver de Keycloak, y qué pasaría si el cliente lo ignorara?

Previene un CSRF de login (inyección de código): un atacante inicia sesión con su propia cuenta y hace que la víctima abra `/auth/callback?code=CODIGO_DEL_ATACANTE`. Si la SPA no compara `state`, canjea ese código y la víctima queda dentro de la cuenta del atacante, donde podría registrar datos o pagos que el atacante luego ve. Con `state`, la SPA solo acepta respuestas a un login que ella misma inició.

3. ¿Por qué un `201` de `POST /api/v1/citas` no prueba que el pago se resolvió?

Porque `citas-ms` responde después de guardar la cita y publicar `cita.creada`, sin esperar a `pago-ms`. El 201 de la cita #21 llegó con pago-ms apagado (Figura 11). El pago se decide después y puede terminar en PAGADA o en CANCELADA.

4. De las tres estrategias de la Tabla 3, ¿cuál elegiste y por qué descartaste las otras dos?

Polling acotado, porque reutiliza el endpoint, la seguridad y el Gateway existentes con un costo bajo. Push se descartó por la infraestructura de conexiones largas autenticadas que exige. La respuesta síncrona se descartó porque volvería a acoplar la creación de la cita a la disponibilidad de pago-ms (sección 5).

5. ¿Por qué configurar CORS en el Gateway y no en cada microservicio?

Porque el navegador solo ve un origen de API (el Gateway). Los microservicios no son accesibles desde el navegador y, si también agregaran cabeceras CORS, la respuesta saldría con `Access-Control-Allow-Origin` duplicado y el navegador la rechazaría. Además, la política queda en un solo lugar: cambiar el dominio del frontend es una propiedad en el Config Server.

6. Si `GET /api/v1/ordenes` no filtra por `idCliente` en el servidor, ¿qué protege el filtrado del frontend y qué no protege?

Solo protege la experiencia: el usuario no ve en pantalla datos ajenos. No protege los datos, porque cualquiera con un token válido puede llamar al endpoint con `curl` y recibir todo. En la veterinaria esto se resolvió en el servidor desde S7: `GET /api/v1/citas` filtra con el `idCliente` del token (`findByIdCliente`) y `GET /citas/{id}` responde 403 si la cita es de otro cliente. La SPA no filtra nada por su cuenta.

09 Reflexión técnica

En el caso de Starbucks (1.6), las tarjetas de regalo permitían transferir saldo varias veces enviando peticiones simultáneas: cada petición leía el saldo antes de que la otra lo descontara. No fallaba la autenticación, porque cada petición venía del dueño legítimo con credenciales válidas. Fallaba la consistencia: dos lecturas del mismo estado tomaban decisiones como si fueran las únicas. Ese es el riesgo que la pantalla "Mis citas" evita del lado del cliente. La SPA nunca muestra "Pagada" por haber recibido un 201, ni deduce el resultado del pago a partir de su propio estado local. Solo informa lo que el servidor confirmó después de que la Saga terminó, y mientras tanto dice "todavía no se sabe". Del lado del servidor, la defensa equivalente ya existe desde S9: el consumidor idempotente con la restricción `uk_pagos_cita_id`, que impide cobrar dos veces la misma cita aunque el evento llegue repetido. Un token válido dice *quién* pide algo. No dice *en qué estado* está el sistema cuando la petición llega.

10 Criterios, limitaciones y bibliografía

Criterio mínimo	Cumplimiento
Nombre de archivo solicitado	✓ `S11_Equipo##_SanchezSaraviaJoaquin.pdf`
Servidor de autorización propio reemplazado por Keycloak	✓ Secciones 2 y 3; auth-ms retirado
PKCE sin Direct access grants	✓ Figura 2 y `unauthorized_client`
Decisión `sub` vs user attribute + mapper, verificada con token real	✓ Sección 3, Figuras 3 y 4, payload
Pantalla asíncrona con estrategia justificada	✓ Sección 5
Camino feliz y de rechazo/fallo reales	✓ Sección 6 (tres escenarios)
Capturas con reloj del sistema y fechas coherentes con los commits	✓ 15:23–15:27, commits 15:10
Error o hallazgo diagnosticado y reflexión	✓ Secciones 7 y 9

Limitaciones conocidas

Registro de clientes. La saga de alta de S9 (auth-ms ↔ cliente-ms) ya no es accesible porque auth-ms se retiró. El siguiente paso es que un evento de registro de Keycloak (o un flujo de administración) cree el perfil y asigne el atributo `idCliente`. **Pago por cita.** `/api/v1/pagos/por-cita/{citaId}` exige rol CLIENTE o ADMIN pero no comprueba que la cita sea del cliente. Se debe aplicar el mismo ABAC de citas-ms comparando `pagos.id_cliente` con el claim. **Sin refresh token.** El token dura 15 minutos y luego el interceptor reinicia el login; la sesión SSO lo hace transparente, pero un refresco silencioso evitaría perder un formulario a medio llenar. **Keycloak en `start-dev`**, sin TLS ni administrador permanente: no apto para producción.

Bibliografía

IETF. RFC 7636 — Proof Key for Code Exchange by OAuth Public Clients (2015). · IETF. RFC 9700 — Best Current Practice for OAuth 2.0 Security (2025). · IETF. OAuth 2.0 for Browser-Based Applications (draft-ietf-oauth-browser-based-apps). · Keycloak Documentation 26.x — Server Administration Guide: user profile, protocol mappers, realm import. · Spring Security Reference — OAuth 2.0 Resource Server (issuer-uri) y CORS. · Angular Documentation v22 — HttpClient interceptors, Router guards, Signals. · Homakov, E. (2015). Hacking Starbucks for unlimited coffee (race condition). · 262dist. Guías S07b y S11 del curso, sistema pagatu.

Anexo: Feedback de la sesión

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?	Que un cliente web no es un `curl` con interfaz. No puede guardar secretos, así que PKCE reemplaza al secreto del cliente, y recibe respuestas que todavía no son definitivas. La interfaz tiene que mostrar ese "todavía no se sabe" en lugar de adivinar el resultado.
¿Qué punto de la clase te resultó más confuso o te dejó con dudas?	Por qué con el Gateway WebMVC `globalcors` no hace nada y CORS se tiene que declarar en Spring Security antes de la autorización. Y la diferencia práctica entre `sub` y un atributo propio en el token.
¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?	¿Cómo se crea el perfil de negocio y se asigna el atributo `idCliente` cuando el usuario se registra directamente en Keycloak (event listener o Admin API)? ¿Conviene un refresh token en una SPA o es mejor el patrón BFF?
Nivel de comprensión de la clase de hoy	✓ ¡Entendido! - Lo domino y podría explicarlo. Más o menos. Necesito ayuda.
¿Cómo puedo ayudarte a comprender mejor el tema?	Con una comparación en vivo entre polling y SSE sobre el mismo Gateway, midiendo peticiones y latencia.
Autoevaluación de participación y esfuerzo	✓ Muy Comprometido/a: Me esforcé al máximo. Comprometido/a. Poco Comprometido/a.
Satisfacción con la clase (1 a 10)	9